Scan Conversion

Here is a description of the data structure and algorithms necessary to do polygon scan conversion.

Edge

An edge structure is needed to hold the all the data associated with an edge that is scan converted. It should contain:

```
int  yLast;     // Final scan line of edge
attr_point  p;   // The values of the edge on this scan line
attr_point  inc; // The incremental changes for the values from
                 // scanline to scanline

Edge * next;     // A pointer to link the edges together in edge tables.
```

The Edge Table

An edge table (an array of Edges) should be dynamically created at the same time the z-buffer is created. The array should have an edge for every scanline of the image. Most of the values in the edges are not used. All that is needed is the pointer to the edges associated with each scan line. These pointers should be initialized to null pointer values. If the scan conversion algorithm is implemented correctly, the initialization should only need to be done once. The edge table should be freed up at the same time the z-buffer is returned to the system memory pool.

`scan_convert()`

This is the main function for scan converting a polygon. It takes an array of attributed points (and most likely an integer indicating the number of attributed points in the polygon). Pseudo-code for the scan conversion is given:

```
scan_convert(an array of attr_points, a count)
{
    Local variables:   An Edge which represents the head of the active
                       edge table (AET)

    if(! buildEdgeList(points, count))
       return;   // No edges cross a scanline

    clear the AET

    Loop an integer, scan, over the scanlines of the display
        {
        Take the edges starting on this scanline from the edge table
        and add them to the active edge table (AET).

        if AET is not empty
           {
              fill between the edge pairs in the AET
              update the AET
              resort the AET
           }
        }
}
```

`buildEdgeList`

This routine takes an array of attributed points and a count and places the edges created by the points into the edge table ET. It returns a flag value indicating whether or not the polygon crossed a scanline.

```
Local variables:   v1 and v2 are indices into the attributed point
array.  v1 is the trailing vertex of the edge.  v2 is the leading
vertex of the edge.  A scanline_crossed variable.

   scanline_crossed = false;


   v1 = count - 1;  // The last vertex in the polygon

   Loop v2 over the number of vertices in the polygon
     {
        if( the y coordinates of v1 and v2 fall on different scanlines)
           scanline_crossed = true;

           if( vertex[v1].y  < vertex[v2].y)
              Make an edge record from vertex[v1] to vertex[v2]
           else
              Make an edge record from vertex[v2] to vertex[v1]

        v1 = v2; // Move to next edge
     }

   return scanline_crossed;
```

makeEdgeRec

This routine takes two attributed points, upper and lower, and inserts an Edge created from those two points into the edge table (ET).

```
Local variables:  two floating point values, dy and factor.  Also a
pointer e for the new Edge created.

    dy = upper.y - lower.y

    Dynamically allocate a new edge.  Pointer to same is stored in e.

    // Calculate the edge value increments between scan lines
    e->inc = (upper - lower) / dy

    // Edge starts on scanline ceil(lower.y)
    factor =  ceil(lower.y) - lower.y  // Gives the fractional position
                                       // of the first scanline crossing

    // Calculate the starting values for the edge
    e->p = lower + factor * e->inc

    // Find the last scanline for the edge
    e->yLast = ceil(upper.y) - 1;

    Insert e into the edge table list of edges starting on scanline
      ceil(lower.y)
```

addActiveList

This routine takes an integer scanline and a pointer to an `Edge`, the AET.

```
    Edge *p, *q;

    p = edge_table[scan].next; // Get the edges starting on this scan line
```

```
while(p)
  {
    q = p->next;  // Hold the rest of the list
    insertEdge(AET, p);
    p = q;
  }

// Keep edge table clean -- edges have been transfered
edge_table[scan].next = 0;
```

insertEdge

This routine takes two pointers to `Edges`, a linked list and an `Edge` e, to be inserted into that list. The list is to be maintained sorted by increasing x coordinate values of the edges in the list.

```
Edge  *p, *q = list;

// p leads
p = q->next;

while(p != 0 && (e->p.x > p->p.x))
  {
    // Step to the next edge
    q = p;
    p = p->next;
  }

// Link the edge into the list after q
e->next = q->next;
q->next = e;
```

updateAET

This function takes an integer scanline and a pointer to an Edge the AET.

```
Edge *q = active, *p = active->next;
// p leads

while(p)
  if(scanline == p->yLast)  // This is the last scanline for this edge
     p = p->next;    // Move p along
     deleteAfter(q)  // get rid of old node
  else
     // Update the attribute values
     p->p += p->inc;

     q = p;
     p = p->next;
```

deleteAfter

This function takes a pointer to an `Edge` q. The node after this `Edge` is to be deleted.

```
Edge *p = q->next;

q->next = p->next;
delete p;
```

resort AET

This function takes a pointer to an Edge, the active edge table, and resorts it by increasing x values. It does this by taking all of the edges out of the active edge list and then reinserting them into the active edge table.

```
    Edge *q, *p = active->next;

    active->next = 0;
    while(p)
      {
        q = p->next;
        insertEdge(active, p);
        p = q;
      }
```

fill between the edges

This function takes a scan line and a pointer to an edge, the active edge table. It fills in the pixels between edge pairs in the table

```
  Edge * p1, *p2;

  p1 = active->next;
  while (p1)
    {
      p2 = p1->next;  // Get the pair of edges from the AET

      if(the x values are not equal)
        {

          // Calculate the attribute increments along the scanline
          dx = p2.x - p1.x
          inc = (p2 - p1) / dx

          // Calculate the starting values for the edge
          factor = ceil(p1.x) - p1.x // Gives the fractional position
                                     // of the first pixel crossing

          value = p1 + factor * inc;


          endx = ceil(p2.x)

          while(value.x < endx)
              Calculate the color for the pixel and plot it.
              x and z come from the current values, y is the current scanline

              // Increment the values
              value += inc;
        }
      p1 = p2->next;
    }
}
```